PROJET DE SÉCURITÉ

REPUBLIQUE TUNISIENNE MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE UNIVERSITÉ TUNIS ELMANAR

FACULTÉ DES SCIENCES DE TUNIS

DEPARTEMENT DES SCIENCES DE L'INFORMATIQUE

Vérification Formelle du Protocole HTTPS avec l'outil UPPAAL

Modélisation du protocole TLS 1.3 (RFC 8446) et analyse des attaques CVE-2014-3566 (POODLE) et

CVE-2014-0160 (Heartbleed)

Réalisé par :

Ousema Hkimi
Spécification

Farah ben Arbi
Modélisation UPPAAL

Baraa Benarfi
Attaque POODLE

Rym Jlidi
Attaque Heartbleed

Amani Arouri
Vérifications formelles

Encadrante : Mme Amina Jandoubi

1) Contexte et Présentation du Protocole HTTPS

a) Introduction à HTTPS:

HTTPS est l'un des protocoles les plus critiques en matière de sécurité, protégeant les sessions bancaires, les authentifications et les communications sensibles. Son historique de vulnérabilités documentées (BEAST, CRIME, POODLE, Heartbleed) en fait un sujet idéal pour la modélisation formelle.

Référence : *RFC 9110 §1 — HTTP Semantics (Introduction)*

b) Propriétés Fondamentales de Sécurité

HTTPS garantit trois propriétés fondamentales de sécurité :

Propriété	Description	Mécanisme	RFC
Confidentialité	Les données échangées sont chiffrées	AES-GCM, ChaCha20-Poly1305	RFC 8446 §9.1
Intégrité	Toute altération du message est détectée	HMAC / AEAD	RFC 8446 §5.2
Authenticité	Le serveur prouve son identité	Certificat X.509 signé par une CA	RFC 5280 §4

Référence : *RFC 8446 §1.3 — The Transport Layer Security Protocol v1.3*

i) Spécifications du Protocole

Ce protocole ne s'agit pas d'un protocole unique, mais d'une pile de protocoles fonctionnant de concert.

Couche	Protocole	Rôle	RFC
Couche Applicative	**HTTP/1.1**	Requêtes/Réponses (GET, POST, codes d'état)	RFC 9110, RFC 9112
Couche Sécurité	**TLS 1.3**	Chiffrement, authentification, intégrité	RFC 8446
Couche Transport	**TCP**	Connexion fiable, ordonnée — Port 443	RFC 9293

ii) Types de messages

- Messages du Handshake TLS 1.3:

Messages envoyés par le Client

- ClientHello: Le client envoie les versions TLS supportées, les suites de chiffrement candidates et sa part de clé publique (KeyShare). C'est le premier message du handshake.

-Finished Client: Un MAC (Message Authentication Code) calculé sur l'ensemble du handshake. Confirme que le client possède les bonnes clés de session.

Messages envoyés par le Serveur

-ServerHello: Le serveur choisit la suite de chiffrement et envoie sa propre part de clé publique (KeyShare). À partir de ce message, les échanges sont chiffrés.

-EncryptedExtensions: Paramètres additionnels chiffrés que le serveur souhaite transmettre confidentiellement (ex : protocoles ALPN supportés).

-Certificate: Le serveur envoie son certificat X.509 (RFC 5280) signé par une Autorité de Certification (CA) pour prouver son identité.

-CertificateVerify: Une signature numérique prouvant que le serveur possède bien la clé privée correspondant au certificat présenté.

-Finished Serveur: Même principe que le Finished client authentifie tout le handshake côté serveur.

Référence : *RFC 8446 §4 — Handshake Protocol (ensemble des messages)*

- Messages de Données Applicatives:

Une fois le handshake terminé, les données HTTP sont échangées chiffrées dans des enregistrements TLS. Les méthodes HTTP principales sont définies dans la RFC 9110 §9 :

-GET: Demander une ressource (page HTML, image, fichier JSON)

- POST: Envoyer des données au serveur (formulaire, authentification)

- PUT: Créer ou remplacer une ressource existante

- DELETE : Supprimer une ressource identifiée par son URI

Référence : *RFC 9110 §9 — Méthodes HTTP / RFC 9112 §2 — Format de message*

iii) Diagramme de Séquence TLS 1.3 — 1-RTT Handshake

CLIENT	MESSAGE / ACTION	SERVEUR
ClientHello	**-> *Versions TLS, suites de chiffrement, clé publique (KeyShare)***	
	<- *ServerHello : paramètres choisis + clé publique serveur*	
	<- *EncryptedExtensions : paramètres chiffrés*	
	<- *Certificate : certificat X.509 du serveur*	
	<- *CertificateVerify : preuve de la clé privée (signature)*	
	<- *Finished : MAC du handshake (authentification)*	
Finished	**-> *MAC client — handshake terminé***	
GET / POST / ...	**-> *Données applicatives HTTP chiffrées (AES-GCM / ChaCha20)***	
	<-*Réponse HTTP chiffrée (200 OK, HTML, JSON, ...)*	

Référence : *RFC 8446 §2 — Overview / §4 Handshake Protocol / Figure 1*

iv) États et Transitions du Protocole HTTPS/TLS 1.3

Le protocole HTTPS suit une machine à états. Chaque connexion passe par les états suivants :

Etat	Description	Déclencheur
IDLE	***Aucune connexion active***	*État initial*
TCP_CONNECTED	***Connexion TCP établie (port 443)***	*TCP SYN/SYN-ACK/ACK*
TLS_HANDSHAKE	***Négociation TLS en cours***	*Envoi du ClientHello*
TLS_ESTABLISHED	***Session sécurisée active***	*Réception du Finished serveur + envoi Finished client*
DATA_TRANSFER	***Échange de données HTTP chiffrées***	*Handshake terminé*
CLOSED	***Connexion fermée***	*Alert close_notify ou erreur fatale*

o Diagramme de transitions :

IDLE → TCP_CONNECTED → TLS_HANDSHAKE → TLS_ESTABLISHED → DATA_TRANSFER

↓ ↓

CLOSED(erreur/timeout) CLOSED

o Transitions d'erreur :

-Si le serveur ne répond pas au ClientHello → timeout → transition vers **CLOSED**

-Si le certificat est invalide → Alert certificate_expired → transition vers **CLOSED**

-Si une erreur fatale survient pendant DATA_TRANSFER → Alert close_notify → **CLOSED**

Référence : RFC 8446 §4 — Handshake Protocol / §6 — Alert Protocol

v) Mécanisme de Réinitialisation en cas d'Erreur

Lorsqu'une erreur fatale survient pendant la session TLS, le protocole suit le mécanisme de réinitialisation suivant défini dans la RFC 8446 §6 :

Étape	Action	RFC
1	Détection de l'erreur (certificat invalide, timeout, données corrompues)	RFC 8446 §6
2	Envoi d'un **Alert fatal** chiffré (ex: certificate_expired, bad_record_mac)	RFC 8446 §6.2
3	**Fermeture immédiate** de la session TLS des deux côtés	RFC 8446 §6.1
4	**Fermeture de la connexion TCP** (FIN/ACK)	RFC 9293
5	**Nouvelle connexion TCP** sur le port 443 si nécessaire	RFC 9293
6	Nouveau handshake TLS 1.3 complet depuis l'état IDLE	RFC 8446 §4

Référence : RFC 8446 §6.1 — Closure Alerts / §6.2 — Error Alerts

Scénario	Comportement TCP	Impact sur TLS
Perte d'un paquet pendant le handshake	TCP retransmet automatiquement	TLS attend — le timer de handshake se déclenche si le délai dépasse T
Perte du ClientHello	TCP retransmet le segment	Le handshake recommence depuis le début
Perte d'un paquet pendant DATA_TRANSFER	TCP retransmet le segment manquant	TLS ne peut pas déchiffrer tant que le flux n'est pas reconstitué
Timeout total (aucune réponse)	TCP abandonne après N tentatives	TLS envoie un Alert close_notify → session fermée → retour à IDLE

vi) Timers et Gestion des Erreurs

La RFC 8446 définit des timers pour éviter le blocage infini du handshake : si le serveur ne répond pas dans un délai T, le système transite vers un état d'erreur ou de réinitialisation.

•Timer de Handshake : délai d'attente entre ClientHello et ServerHello

•Timer de Session : durée de validité d'une session TLS (New Session Ticket)

•Alert Protocol : messages d'erreur chiffrés (close_notify, certificate_expired...)

Référence : *RFC 8446 §6 — Alert Protocol / §4.6.1 New Session Ticket*

vii) Qualité de Service (QoS)

La QoS dans HTTPS est principalement mesurée par la latence d'établissement de connexion.
-TLS 1.3 améliore significativement la QoS par rapport à TLS 1.2 :

-TLS 1.2 : 2-RTT (deux allers-retours) pour établir la session → latence élevée

-TLS 1.3 : 1-RTT (un seul aller-retour) → latence réduite de 50%

-TLS 1.3 0-RTT : reprise de session sans aller-retour supplémentaire (RFC 8446 §2.3)

Référence : ***RFC 8446 §2.3 — 0-RTT Data / Introduction §1.2 — Améliorations TLS 1.3***

2) Modélisation UPPAAL :

le modèle formel du protocole TLS 1.3 (RFC 8446) construit dans UPPAAL est composé de deux templates principaux Client et Server qui interagissent via des canaux synchronisés pour reproduire fidèlement le handshake TLS 1.3 dans sa forme nominale (sans attaquant).il est composé de 2 automates:

- Client
- Server

-Les échanges sont synchronisés via des canaux UPPAAL

a) Template Client

-Le template **Client** modélise le comportement d'un navigateur ou d'une application cliente initiant une connexion HTTPS. Il déclenche le handshake TLS 1.3, reçoit et valide la réponse du serveur, et établit la session sécurisée.

i) États du template Client

État (id UPPAAL)	Nom	Description
id0	Init	État initial — aucune connexion en cours
id1	HelloSent	ClientHello envoyé, attente réponse serveur. Invariant : c_timer <= TIMEOUT
id2	ServerHelloReceived	ServerHello reçu, prêt à envoyer ClientFinished
id3	Connected	Handshake complet — session_established = true, session_client = true
id4	DataTransfer	Échanges de données applicatives chiffrées via ch_appData
id5	Error	État d'erreur — alerte reçue ou envoyée
id6	Closed	Connexion fermée — timeout ou fermeture propre. Retour possible vers Init

ii) Transitions principales du template Client

Transition	Canal	Garde / Action	Description
Init → HelloSent	ch_clientHello!	c_timer = 0	Envoi ClientHello, démarrage du timer. RFC 8446 §4.1.2
HelloSent → ServerHelloReceived	ch_serverHello?	(aucune garde)	Réception ServerHello + Certificate + Finished du serveur. RFC 8446 §4.1.3
ServerHelloReceived → Connected	ch_clientFinished!	session_client=true, session_established=true	Envoi ClientFinished, session établie. RFC 8446 §4.4.4
Connected → DataTransfer	ch_appData!	(aucune garde)	Début échanges applicatifs chiffrés. RFC 8446 §4.6
DataTransfer → Error	ch_alert!	(aucune garde)	Alerte fatale — fermeture immédiate de la session
HelloSent → Closed	ch_alert!	c_timer >= TIMEOUT	Timeout handshake — fermeture par dépassement de TIMEOUT=30. RFC 8446 §6
Closed → Init	(interne)	(aucune garde)	Réinitialisation — possibilité de nouvelle session (boucle)

b) Template Server

Le template **Server** modélise un serveur web acceptant des connexions HTTPS. Il attend le **ClientHello**, envoie les messages d'authentification (ServerHello, Certificate, CertificateVerify, Finished), puis établit la session sécurisée après réception du **ClientFinished**.

i) États du template Server

État (id UPPAAL)	Nom	Description
id16	**Init**	État initial — serveur en écoute. Boucle possible via ch_alert! (self-loop)
id17	**HelloReceived**	ClientHello reçu, négociation démarrée. version_negotiated = 3, c_timer = 0
id18	**HelloSent**	ServerHello + Certificate + EncryptedExtensions envoyés. Attente ClientFinished
id19	**FinishedReceived**	ClientFinished reçu. Transition interne vers Session (sans canal)
id20	**Session**	Session TLS 1.3 active. session_established = true côté serveur
id21	**HR_Computing**	Calcul de la reponse Heartbeat. Evaluation de memory_leak = (heartbeat_length > payload_size)
id22	**Data**	Échanges de données applicatives chiffrées via ch_appData?
id23	**HR_Computing_Data**	Préparation et émission de ch_heartbeatResp!
id24	Closed	Session fermée proprement. Retour vers Init possible

ii) Transitions principales du template Server

Transition	Canal	Action	Description
Init → HelloReceived	ch_clientHello?	version_negotiated=3, c_timer=0	Réception ClientHello, init TLS 1.3. RFC 8446 §4.1.2
HelloReceived → HelloSent	ch_serverHello!	(aucune action)	Envoi ServerHello + Certificate + Extensions. RFC 8446 §4.1.3
HelloSent → FinishedReceived	ch_clientFinished?	(aucune action)	Réception ClientFinished. RFC 8446 §4.4.4
FinishedReceived → Session	(interne)	(aucune action)	Transition interne — session établie côté serveur
Session → Data	ch_appData?	(aucune garde)	Réception données applicatives. RFC 8446 §4.6
Data → Data	ch_appData?	(aucune action)	Réception continue de données applicatives
Data → Closed	ch_alert?	(aucune action)	Fermeture propre sur réception d'alerte close_notify. RFC 8446 §6.1
Session → Closed	ch_alert?	(aucune action)	Réception d'une alerte — fermeture de la session
Closed → Init	(interne)	(aucune action)	Réinitialisation — prêt pour une nouvelle connexion

c) Propriétés vérifiées sur le modèle nominal

Propriété CTL	Requête CTL	resultat	Stratégie
Absence de deadlock	A[] not deadlock	SATISFAIT	BFS
Session établissable	E<> session_established	SATISFAIT	DFS
Le client atteint Connected	E<> Client.Connected	SATISFAIT	DFS
Le serveur reçoit ClientFinished	E<> Server.FinishedReceived	SATISFAIT	BFS
Le serveur atteint une session	E<> Server.Session	SATISFAIT	DFS

Avant l'intégration des attaquants, les propriétés de base ont été vérifiées sur le modèle nominal (Client + Server), avec les résultats suivants.

d) Stratégies d'exploration utilisées

Pour vérifier ces propriétés, nous avons utilisé différentes stratégies d'exploration proposées par UPPAAL via l'outil verifyta.

Les commandes utilisées sont :

verifyta.exe -t 0 -o 0 model_nominal.xml (BFS + trace essentielle)

verifyta.exe -t 1 -o 1 model_nominal.xml (DFS + trace la plus courte)

Différentes stratégies d'exploration ont été utilisées dans UPPAAL. La recherche en largeur (BFS) permet de détecter rapidement les violations de propriétés de sécurité en trouvant le plus court contre-exemple, tandis que la recherche en profondeur (DFS) est plus efficace pour vérifier l'atteignabilité des états.

3) Modélisation des attaques CVE

a) Attaque de spécification : POODLE (CVE-2014-3566)

i) Description (idée générale):

POODLE (*Padding Oracle On Downgraded Legacy Encryption*) est une vulnérabilité de sécurité révélée en octobre 2014 par des chercheurs de Google, notamment Bodo Möller, Thai Duong et Krzysztof Kotowicz. Elle provient d'une faiblesse de conception du protocole SSL 3.0, en particulier dans l'utilisation du mode de chiffrement CBC sans vérification correcte du padding.

Cette vulnérabilité devient exploitable lorsqu'un attaquant est capable de réaliser une attaque de type *Man-In-The-Middle* (MITM) et de forcer une rétrogradation de protocole (*downgrade attack*). En perturbant la négociation TLS, l'attaquant peut contraindre le client et le serveur à abandonner les versions récentes et sécurisées (TLS 1.0, TLS 1.1, TLS 1.2) au profit de SSL 3.0, encore maintenu pour des raisons de compatibilité.

Une fois SSL 3.0 activé, l'attaquant peut exploiter une faiblesse appelée **oracle de padding**, qui lui permet d'analyser les réponses du serveur et d'extraire progressivement des informations sensibles, telles que des cookies de session.

Références :

- Google Security Blog (2014), *This POODLE bites: exploiting SSL 3.0* https://security.googleblog.com/2014/10/this-poodle-bites-exploiting-ssl-30.html
- CVE-2014-3566 (MITRE) https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-3566

ii) Modélisation UPPAAL

Template : Attacker_POODLE

États :idle, inject, retry, downgrade, downgrade_achieved, padding_oracle, session_decrypted

Scénario modélisé :

1. L'attaquant POODLE commence dans l'état initial **idle**, où il attend le lancement de l'attaque.

2. Il passe ensuite dans l'état **inject**, où il initie la tentative d'attaque en perturbant la négociation TLS et en forçant un processus de rétrogradation du protocole.

3. L'attaquant entre ensuite dans l'état **retry**, où il répète les tentatives de négociation tant que la condition de succès n'est pas atteinte.

4. À chaque itération, la variable **version_negotiated** est décrémentée afin de simuler la réduction progressive du niveau de sécurité du protocole : `version_negotiated=version_negotiated−1`

5. Tant que **version_negotiated > 0** et que le nombre de tentatives reste inférieur à **MAX_RETRY**, l'attaquant boucle entre les états **retry** et **inject**, simulant ainsi le processus de downgrade progressif.

6. Lorsque **version_negotiated = 0**, l'attaquant atteint l'état **downgrade**, ce qui correspond à une rétrogradation réussie vers une version faible du protocole (SSL 3.0 dans le modèle).

7. Il passe ensuite dans l'état **downgrade_achieved**, où la vulnérabilité devient exploitable.

8. L'attaquant passe ensuite à l'état **padding_oracle**, où il exploite la vulnérabilité de type *padding oracle*, en analysant les réponses du serveur afin d'extraire progressivement des informations sur les données chiffrées.

9. Enfin, il atteint l'état **session_decrypted**, qui représente la réussite complète de l'attaque avec le déchiffrement de la session TLS.

iii) Requêtes de vérification

Résultats :

- E<> Attacker_POODLE.downgrade=> SATISFAIT
- E<> Attacker_POODLE.session_decrypted => SATISFAIT

Le modèle permet encore la rétrogradation vers SSL 3.0 et la réussite potentielle de l'attaque POODLE avant application de la contre-mesure.

iv) Contre-mesure proposée

Contre-mesure : imposer une version minimale TLS 1.2 dans la garde de la transition ServerHello (HelloReceived → HelloSent) du template Server :

Garde ajoutée : version_negotiated >= 2 (TLS 1.2 minimum)

Apres ajout de cette garde :

A[]notAttacker_POODLE.downgrade_achieved=>SATISFAIT
A[] not Attacker_POODLE.session_decrypted => SATISFAIT

Conclusion :
La contre-mesure est validée formellement : le downgrade vers SSL 3.0 est rendu impossible dans le modèle.

b) Attaque d'implémentation : Heartbleed (CVE-2014-0160)

i) Description (idée générale)

L'attaque Heartbleed (CVE-2014-0160) est une vulnérabilité critique découverte en avril 2014 dans la bibliothèque OpenSSL (versions 1.0.1 à 1.0.1f).
Elle exploite un défaut dans l'extension Heartbeat définie par la RFC 6520, utilisée pour vérifier la continuité des connexions TLS.

Cette extension permet l'échange de messages contenant :

- un payload (données)
- une longueur déclarée (heartbeat_length)

-Le problème principal est l'absence de vérification entre la taille réelle du payload et la taille déclarée.

Un attaquant peut envoyer une requête malformée (ex : heartbeat_length > payload réel) provoquant une lecture non autorisée de la mémoire du serveur.
Cela peut exposer des données sensibles comme des clés privées ou mots de passe, violant ainsi la confidentialité.

ii) Modélisation UPPAAL

Le modèle UPPAAL inclut un composant **Attacker_Heartbleed** représentant l'exploitation de la vulnérabilité.

États de l'attaquant :

- idle
- session_active
- send_heartbeat
- wait_response
- leak
- success

✔️ Scénario modélisé : Dans le modèle, l'attaque Heartbleed est représentée comme suit :

1️⃣ Initialisation

L'attaquant attend qu'une session TLS soit établie :

- session_established = true

2️⃣ Envoi de la requête malformée

L'attaquant envoie une requête Heartbeat via :

- ch_heartbeatReq!

3️⃣ Traitement côté serveur

Le serveur reçoit la requête et exécute un calcul interne :

memory_leak = (heartbeat_length > payload_size)

4️⃣ Réponse du serveur

Le serveur répond via :

- ch_heartbeatResp!

5️⃣ Observation de la fuite

L'attaquant observe uniquement la variable globale :

- memory_leak
- Si : memory_leak == true → transition vers l'état **leak**
- sinon → retour à idle (réessai)

6 Succès de l'attaque

Si la fuite est détectée, l'attaquant atteint l'état :

- **success**

🔸 **Variables utilisées**

- heartbeat_length : taille déclarée par l'attaquant
- payload_size : taille réelle côté serveur
- memory_leak : indique si une lecture mémoire invalide a eu lieu
- session_established : indique une session TLS active
- hb_success : indique la réussite finale de l'attaque

iii) Vérification (UPPAAL)

Absence de deadlock :A[] not deadlock -> SATISFAIT

Le système ne contient aucun blocage.

Atteignabilité de la fuite mémoire :E<> memory_leak → **SATISFAIT**

L'attaque Heartbleed est réalisable dans le système.

Propriété de sécurité :A[] not memory_leak → **VIOLÉ**

Le système permet une fuite mémoire, ce qui confirme la vulnérabilité.

Fuite pendant session active :E<> (memory_leak && session_established) → **SATISFAIT**

La fuite peut se produire durant une session TLS valide.

Vérification probabiliste (UPPAAL-SMC) :

Pr[<=1000] (<> Attacker_Heartbleed.success)//uppaal-smc

`Résultat : proche de 1 (≈ 0.99)`

L'attaque réussit avec une probabilité très élevée.

iv) Contre-mesure

Pour corriger cette vulnérabilité, il est nécessaire d'ajouter une vérification stricte dans le serveur :

hb_payload_size <= hb_real_size

Cette condition empêche toute lecture hors limites de mémoire.

Après correction

A[] not hb_leak ->**SATISFAIT**

E<> memory_leak → **VIOLÉ:** L'attaque devient impossible après correction.

v) Conclusion

L'attaque Heartbleed est une vulnérabilité critique causée par l'absence de vérification de la taille du payload dans le protocole TLS.

La modélisation UPPAAL a permis de démontrer que :

- le système fonctionne correctement (pas de deadlock)
- mais il est vulnérable à une fuite mémoire

L'ajout d'une vérification stricte empêche totalement l'exploitation de la faille.

4) Synchronisation du Système et Logique CTL

a) Types de canaux UPPAAL

Canal	Type	Usage
ch_clientHello, ch_serverHello, ch_clientFinished	Binaire	Handshake TLS (ordre strict)
ch_appData, ch_alert	Broadcast	Données et alertes (non bloquant)
ch_heartbeatReq, ch_heartbeatResp	Binaire	Interaction Heartbleed

- Les canaux binaires exigent un émetteur et un récepteur simultanément → ordre strict garanti
- Les canaux broadcast permettent l'envoi sans attendre un récepteur → le client n'est jamais bloqué sur une alerte

b) Composition parallèle

Les quatre automates s'exécutent en parallèle selon la sémantique entrelacée d'UPPAAL.
L'espace d'états global = produit cartésien des états individuels × valuations des variables × régions d'horloges.
UPPAAL explore cet espace exhaustivement lors de la vérification.

c) Logique CTL

CTL (Computation Tree Logic) évalue les propriétés sur l'arbre de toutes les exécutions possibles.

Forme	Lecture	Signification
A[] P	P vrai dans tout état atteignable	Sûreté
E<> P	Il existe un chemin où P devient vrai	atteignabilité
A<> P	P finit par être vrai sur tout chemin	vivacité (progression garantie)
E[] P	Il existe un chemin où P est toujours vrai	Invariant existentiel

➔ A[] et E<> sont les deux formes utilisées dans ce projet.

d) Les Septs Requêtes CTL

Le modèle contient 7 requêtes couvrant absence de blocage, atteignabilité et sécurité.

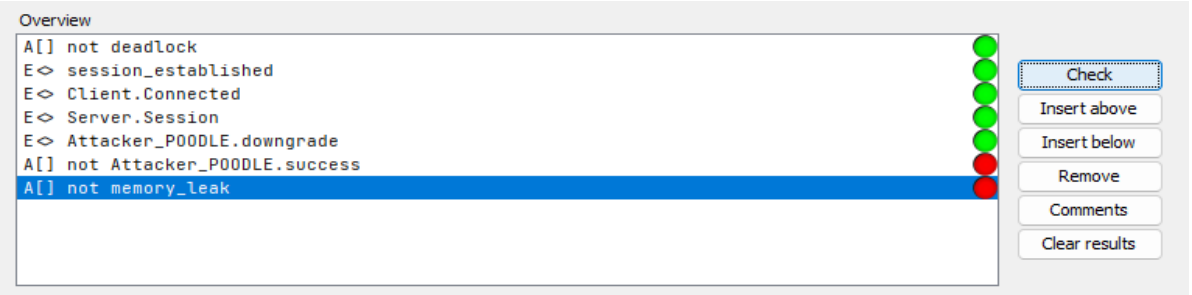
- **Q1 — A[] not deadlock** → absence d'interblocage global
- **Q2 — E<> session_established** → session TLS possible
- **Q3 — E<> Client.Connected** → client connecté atteignable
- **Q4 — E<> Server.Session** → session serveur atteignable
- **Q5 — E<> Attacker_POODLE.downgrade** → attaque POODLE atteignable
- **Q6 — E<> Attacker_POODLE.session_decrypted** → atteignabilité de la compromission POODLE

- **Q7 — A[] not memory_leak** → sûreté Heartbleed

e) Résultats de Vérification CTL

Les résultats de la vérification formelle permettent d'analyser le comportement global du système et de distinguer les propriétés satisfaites des vulnérabilités structurelles.

#	Formule	Attendu	obtenue	stratégie
Q1	A[] not deadlock	SATISFAIT	✅ SATISFAIT	BFS
Q2	E<> session_established	SATISFAIT	✅ SATISFAIT	DFS
Q3	E<> Client.Connected	SATISFAIT	✅ SATISFAIT	DFS
Q4	E<> Server.Session	SATISFAIT	✅ SATISFAIT	DFS
Q5	E<> Attacker_POODLE.downgrade	SATISFAIT	✅ SATISFAIT	DFS
Q6	Q6 | E<> Attacker_POODLE.session_decrypted	NON SATISFAIT	❌ NON SATISFAIT	BFS
Q7	A[] not memory_leak	NON SATISFAIT	❌ NON SATISFAIT	BFS



f) Analyse des résultats

Analyse des propriétés satisfaites (Q1–Q5)

Les propriétés Q1 à Q5 sont validées par le modèle.
Cela confirme que :

- le système est exempt de blocage global grâce à la structure des synchronisations et des états intermédiaires,
- le protocole TLS modélisé permet l'établissement d'une session valide,
- les chemins d'exécution client/serveur sont correctement réalisables via les canaux synchronisés,
- l'attaquant POODLE dispose d'un chemin d'exécution lui permettant d'atteindre un état de rétrogradation.

Analyse des propriétés non satisfaites (Q6–Q7)

Les propriétés de sûreté Q6 et Q7 sont invalidées.

- Pour Q6, l'état Attacker_POODLE.success est atteignable dans certaines exécutions, ce qui met en évidence une faiblesse structurelle du protocole face à la rétrogradation.
- Pour Q7, la condition de fuite mémoire associée à Heartbleed est satisfaite dans le modèle, notamment lorsque la condition heartbeat_length > payload_size entraîne systématiquement une écriture de memory_leak.

g) Stratégies d'exploration

- **BFS (Breadth-First Search)** : utilisée pour les propriétés de sûreté afin d'obtenir des contre-exemples minimaux.
- **DFS (Depth-First Search)** : utilisée pour les propriétés d'atteignabilité afin d'explorer efficacement les chemins du système.
- **Random DFS** : utilisée pour explorer des comportements non déterministes et compléter l'analyse.

h) Analyse des Deadlocks

Un deadlock en UPPAAL est un état global où aucun automate n'a de transition activée → le système est bloqué définitivement

Sources de deadlock corrigées :

Source	État concerné	Type de correction
1	Attacker_POODLE.inject	Pas de blocage : transition vers retry si version_negotiated > 0, sinon progression via downgrade jusqu'à session_decrypted
2	Attacker_POODLE.success	Self-loop tau
3	Attacker_Heartbleed.success	Self-loop tau
4	Server.Session	Nouvelle transition ch_alert?

- Après toutes les corrections, Q1 A[] not deadlock retourne **SATISFAIT** en BFS

5) Vérification Probabiliste (UPPAAL-SMC)

a) Statistical Model Checking

La vérification CTL classique repose sur une exploration exhaustive de l'espace d'états et produit un résultat binaire (SATISFAIT / NON SATISFAIT).

UPPAAL-SMC (Statistical Model Checking) adopte une approche différente :
le système est simulé un grand nombre de fois de manière aléatoire afin d'estimer la probabilité qu'une propriété soit satisfaite, accompagnée d'un intervalle de confiance.

b) Comparaison CTL vs SMC

Critère	CTL classique	UPPAAL-SMC
Type de résultat	SATISFAIT / NON SATISFAIT	Probabilité [0,1]
Méthode	Exploration exhaustive	Simulation statistique
Nature	Déterministe	Stochastique
Garantie	Preuve formelle	Estimation probabiliste

c) Requête utilisée

Requête utilisée : **Pr[<=1000](<> Attacker_Heartbleed.success)**

Cette propriété exprime la probabilité que l'attaquant atteigne l'état *success* dans un délai inférieur ou égal à 1000 unités de temps.

d) Résultats SMC

Scénario	Propriété	Résultat
Avant contre-mesure	**Pr[<=1000](<> Attacker_Heartbleed.success)**	**≈ 0.99**
Avant contre-mesure	**Pr[<=1000](<> memory_leak)**	**≈ 0.99**
Après contre-mesure	**Pr[<=1000](<> Attacker_Heartbleed.success)**	**≈ 0.00**
Après contre-mesure	**Pr[<=1000](<> memory_leak)**	**≈ 0.00**

e) Interprétation

Les résultats montrent que :

- `Avant correction, l'attaque Heartbleed est presque certaine (probabilité ≈ 0.99), ce qui confirme un risque critique.`
- Après contre-mesures, la probabilité chute à 0.00, indiquant l'élimination du scénario d'attaque dans les simulations.

Ainsi, la vérification SMC complète la vérification CTL en apportant une quantification probabiliste des risques.

Les résultats obtenus avec UPPAAL-SMC confirment et renforcent les conclusions de la vérification CTL en montrant l'impact des vulnérabilités avant et après correction.

f) Stratégies UPPAAL

Stratégie	Option
BFS	-t 0 -o 0
DFS	-t 0 -o 1
Random DFS	-t 0 -o 2

6) Discussion Sécurité et Limites

a) Analyse de sécurité

POODLE (CVE-2014-3566)

Dans le modèle, la vulnérabilité provient du fait que le serveur accepte toute version du protocole sans vérification stricte lors du ServerHello. Cela permet à l'attaquant de forcer une rétrogradation progressive jusqu'à SSL 3.0 via la variable version_negotiated.

Contre-mesure proposée :

Ajout d'une contrainte de type garde sur l'état *HelloReceived* → *HelloSent* imposant :

`version_negotiated ≥ 2`, empêchant ainsi la descente vers des versions vulnérables.

Heartbleed (CVE-2014-0160)

La fuite mémoire apparaît lorsque le serveur accepte une requête Heartbeat sans validation des tailles. Dans le modèle, la condition heartbeat_length > payload_size entraîne systématiquement une fuite mémoire.

Contre-mesure proposée :
Ajout d'une garde sur l'état *HB_Computing* imposant :
`heartbeat_length ≤ payload_size`, empêchant toute condition de dépassement.

b) Résultats après contre-mesures

Requête	Avant	Après
Q6 — A[] not Attacker_POODLE.success	NON SATISFAIT	SATISFAIT
Q7 — A[] not memory_leak	NON SATISFAIT	SATISFAIT

c) Limites du modèle

Malgré la validation des propriétés, plusieurs simplifications doivent être soulignées :

- le modèle est abstrait et ne représente pas une implémentation cryptographique réelle (AES, RSA, etc.),
- le handshake TLS est fortement simplifié (réduction à quelques canaux),
- un seul attaquant est modélisé par vulnérabilité, sans interactions multi-attaquants,
- les paramètres heartbeat_length et payload_size sont statiques et non dynamiques,
- l'attaque POODLE ne simule pas un véritable scénario Man-in-the-Middle.

d) Perspectives d'amélioration

Plusieurs extensions peuvent être envisagées :

- intégration d'autres attaques (BEAST, CRIME, DROWN),
- introduction de plusieurs clients simultanés pour analyser l'isolation des sessions,
- utilisation avancée de UPPAAL-SMC pour des analyses probabilistes plus fines,
- ajout de comportements dynamiques pour les paramètres du Heartbeat.

7) Répartition des tâches et contributions

a) Vue d'ensemble

Projet réalisé par 5 membres sous encadrement de Mme Amina Jandoubi (Université Tunis El Manar).

Organisation progressive : spécification → modélisation → attaques → vérification → intégration.

b) Tableau des contributions

Membre	Tâches réalisées	Livrables

M1	**Introduction, contexte, analyse des RFC (TLS 1.3, HTTP, X.509), diagramme de séquence TLS 1.3, coordination globale**	**Sections 1–2 + diagramme**
M2	**Templates Client et Server, synchronisation, horloges, construction du modèle nominal**	**model_nominal.xml**
M3	**Modélisation CVE-2014-3566, automate Attacker_POODLE, contre-mesure version_negotiated ≥ 2, vérification CTL**	**attack_poodle.xml + Section 5.1 +Sections 3–4**
M4	**Modélisation CVE-2014-0160, analyse de fuite mémoire, contre-mesure heartbeat_length ≤ payload_size**	**attack_heartbleed.xml + Section 5.2**
M5	**Cohérence globale, vérification CTL/SMC, intégration des modèles, analyse des résultats et rédaction finale du rapport**	**Sections 6–9 + rapport final**

c) Résultats principaux

Les résultats obtenus montrent que :

- Q1 à Q5 sont SATISFAITES (absence de deadlock, cohérence TLS, atteignabilité correcte)
- Q6 et Q7 sont NON SATISFAITES, révélant les vulnérabilités initiales
- après contre-mesures, Q6 et Q7 deviennent SATISFAITES
- SMC : probabilité ≈ 0.99 → ≈ 0.00 après correction

d) Apports du projet

- détection formelle de vulnérabilités réelles
- validation rigoureuse de contre-mesures
- combinaison CTL + SMC pour analyse qualitative et quantitative
- rôle clé de UPPAAL dans l'exploration exhaustive et la génération de contre-exemples

e) Conclusion générale

Ce projet montre que la vérification formelle avec UPPAAL permet une analyse rigoureuse des protocoles critiques comme HTTPS/TLS. Elle fournit des garanties fortes sur la sécurité du système et complète efficacement les méthodes de test classiques.